

Bài 12.
Các thuật toán tìm kiếm

Nội dung

I. Bài toán tìm kiếm
II. Thuật toán tìm kiếm tuần tự
III. Thuật toán tìm kiếm nhị phân trên mảng

I. Bài toán Tìm kiếm- Search

Input: Cho một tập ***S*** các phần tử, mỗi phần tử là một bộ gồm khóa-giá trị (key-value) và một khóa ***k*** bất kỳ.

Output: Trong ***S*** có phần tử có khóa ***k*** hay không?

Khóa:

- Là thành phần (trường) dữ liệu để tìm kiếm
- Các khóa phải thỏa mãn một quan hệ thứ tự tổng thể.
- So sánh được xác định cho bất kỳ cặp khóa nào và quy tắc so sánh phải là phản xạ, phản đối xứng và bắc cầu.

Các phương pháp tìm kiếm

- Tìm kiếm tuần tự (sequence search)
- Tìm kiếm nhị phân (Binary search)
- Bảng băm (hash table)

II. Tìm kiếm tuần tự

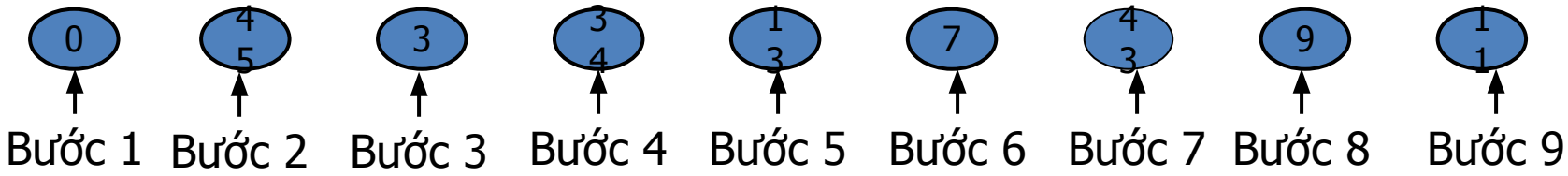
❑ **Tập *S* các phần tử được lưu bằng mảng hoặc danh sách liên kết.**

- Thuật toán tìm kiếm:

- Xuất phát từ phần tử đầu của dãy, thực hiện so sánh khóa của nó với ***k***. Nếu trùng nhau thì dừng lại, nếu không trùng thì lặp lại với phần tử tiếp theo.
- Quá trình dừng lại khi tìm thấy hoặc không còn phần tử nào nữa. Khi đó thông báo không tìm thấy.

Ví dụ 1

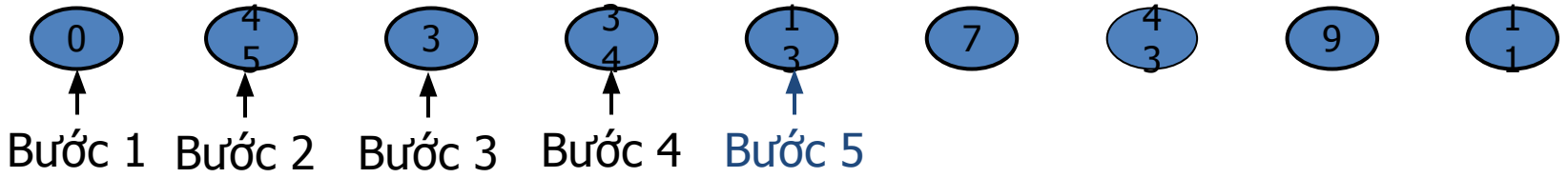
- Cho dãy S: 0 4 5 3 3 4 1 3 7 4 3 9 1 1
- Tìm xem trong dãy có phần tử k=33
- Quá trình tìm kiếm



- Không tìm thấy

Ví dụ 2

- Cho dãy S: 0 4 5 3 3 4 1 3 7 4 3 9 1 1
- Tìm xem trong dãy có phần tử k=13
- Quá trình tìm kiếm



- Tìm thấy, tại vị trí 5

Thuật toán

Input: Cho một dãy *S* các phần tử, mỗi phần tử là một bộ *key* và *value*.
Một khóa *k* bất kỳ.

Output: Trong *S* có phần tử có khóa *k* hay không?

```
 i = 0;
while (i<n){
    if (S[i].key == k)
      return i;
    i =  i+1; //Chuyển sang phần tử kế tiếp
}
return -1;
```

Thời gian chạy

- Trong trường hợp xấu nhất thuật toán phải duyệt qua tất cả các phần tử của S.
- Vậy thời gian chạy là O(n)

III. Tìm kiếm nhị phân trên mảng



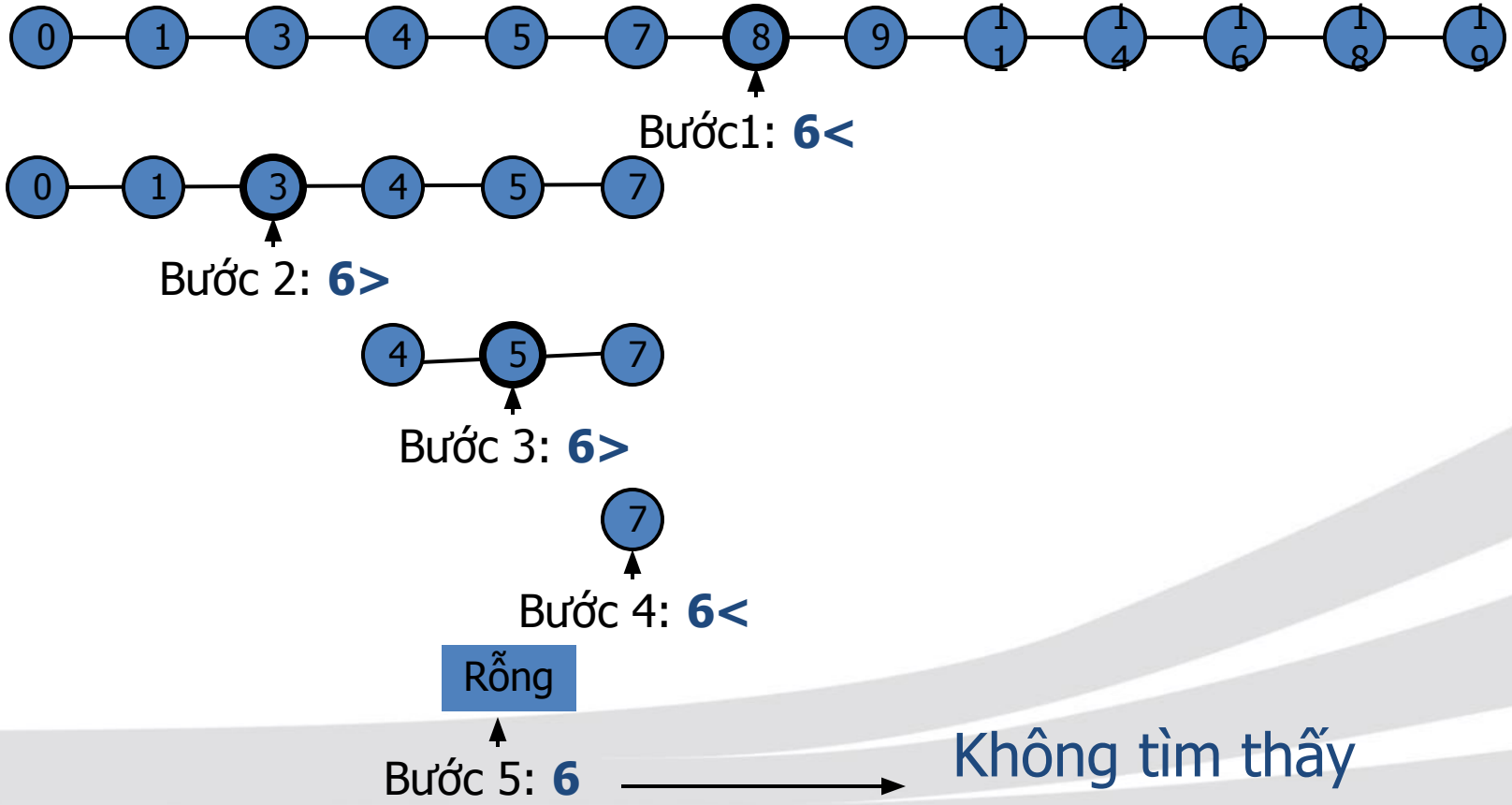
- **Các phần tử của S được lưu trữ trong mảng và được sắp xếp theo thứ tự tăng dần (giảm dần) của giá trị khóa (key).**

- Thuật toán tìm kiếm nhị phân được thiết kế dựa trên chiến lược loại trừ

- Thuật toán: So sánh khóa ***k*** với khóa của phần tử ở giữa dãy.
  - Nếu trùng thì thông báo tìm thấy và dừng
  - Nếu k> thì gọi đệ qui tìm trên nửa cuối dãy
  - Nếu k< thì gọi đệ qui tìm trên nửa đầu dãy
  - Quá trình tìm nếu phải tìm trong dãy rỗng thì dừng lại và thông báo không tìm thấy

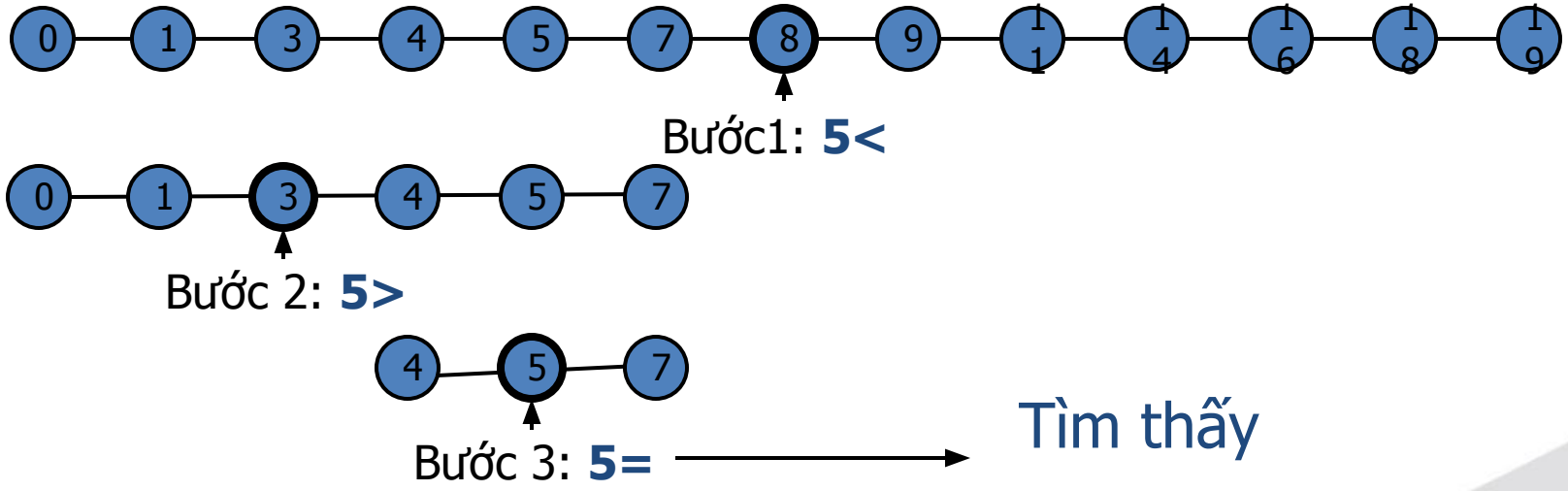
Ví dụ 1

- Cho dãy dưới đây. Tìm phần tử k=6



Ví dụ 2

- Cho dãy dưới đây. Tìm phần tử k=5



Thuật toán

```
Algorithm BinarySearch(S, k, n);
    i = 1;
    j = n;
    while(i<=j){
        mid = (i+j) / 2;
        if  (k==S[mid].Key
          return mid;
       else
            if  (k < S[mid].Key)
             j = mid – 1;
            else
             i = mid+1;
    }
    return -1;
```

Thời gian chạy

- Sau mỗi lần tìm kiếm, thì dãy cần tìm lại được chia đôi và ta chỉ phải tìm trên một nửa
- Trong trường hợp xấu nhất là không tìm thấy phần tử có khóa k. Và như vậy ta phải thực hiện chia đôi liên tiếp đến khi được dãy rỗng. Số lần thực hiện chia đôi là: $\log_2 n$
- Vậy thời gian chạy là $O(\log_2 n)$

Hết